Cleveland 311: Are Requests Closed On Time?

Project documentation

Arockia Robin Sebastin Leo · October 2026

1. Background

Cleveland residents use 311 to report non-emergency problems such as missed trash pickups, potholes, illegal dumping and vacant properties. Each request gets a **target date**, the date the City aims to resolve it by.

The Mayor's Strategic Plan lists **"% of 311 service requests closed on time"** as a City performance measure. The City's public 311 dashboard shows request volume, status, service type and location, with a default view of the last month.

Project question: using the City's open data, how often are 311 requests closed by their target date, and where does that vary by department, request type, ward and time?

Audience: analysts and managers who track City service performance, and department data leads who own the source data.

Scope: public data only. This is an independent analysis, not the City's official calculation.

2. Data

Item	Detail
Source	City of Cleveland Open Data Portal: 311 Service Requests
Downloaded	Oct 24, 2026, CSV, 64 MB
Coverage	188,406 requests, Feb 28, 2024 to Oct 23, 2026
Columns used	Reference number, service category, request type, responsible agency and division, status, requested / closed / target dates, request source, neighborhood, 2026 ward, latitude, longitude

3. Methodology

3.1 Pipeline

CSV → `raw_311` → `requests_clean` → validation checks → analysis → `requests_bi` → dashboard

1. **Load.** Imported the CSV unchanged into SQLite as `raw_311`, so the original is always available.
2. **Clean** (`sql/01_clean_311.sql`). Built `requests_clean` with short lowercase column names, trimmed text, real NULLs instead of empty strings, numeric ward and coordinates, and ISO dates.
3. **Validate** (`sql/02_data_checks.sql`). Five checks: row count and date range, missing values, duplicates, impossible dates, and status consistency.

4. **Analyze** (`sql/03_analysis.sql`). On-time rate overall and by agency, ward, request type and year; average days to close; open requests past their target.
5. **Prepare for BI** (`sql/04_bi_table.sql`). One table with `valid_for_sla`, `on_time` and `days_to_close` flags.
6. **Visualize** (`python/dashboard.py`). One-page dashboard image from the SQL results.

3.2 Definitions

Term	Definition
On time	`closed_at <= target_at`
Included in on-time rate	Closed, has a target date, and `closed_at >= requested_at` (179,842 requests)
Days to close	`julianday(closed_at) - julianday(requested_at)`
Ward	2026 ward boundaries

4. Results

4.1 Data quality

Check	Result	Decision
Row count	188,406 in, 188,406 out	No rows lost in cleaning
Duplicates	0 duplicate reference numbers	None needed
Status vs. closed date	Every Closed request has a closed date; no Open request does	None needed
Closed before requested	915 rows: Public Works 567, Parks & Recreation 348; 739 from inbound calls, about 2 hours apart	Excluded. The pattern suggests requests logged after crews finished the work. Worth confirming with department data leads.
No target date	3,660 rows	Excluded from on-time rate
No ward	429 rows	Excluded from ward results only
Open and past target	2,689 requests	Reported separately; the rate counts closed requests only

4.2 Performance

Measure	Result
Overall on-time rate	**68.1%**
Average days to close	16.6 days
On-time, Jan–Sep of each year	53.2% (2024), 65.6% (2025), 73.6% (2026)
By agency	Parks & Recreation 73.6%, Public Works 69.5%, Building & Housing 57.7%

Measure	Result
Building & Housing timing	51.8 days to close on average, against a 22-day average target window
Highest on-time types (1,000+ requests)	Missing Recycling Welcome Packet 99.9%, Missed Refuse Pick-up 98.0%, Dead Animal Pickup 96.4%
Lowest on-time types (100+ requests)	Electrical Issue 9.5%, No Water/No Hot Water 14.6%, Graffiti Removal 21.6%, Pothole Repair 25.9%, Vacant Property 34.3%
By 2026 ward	64.5% (Ward 8) to 72.1% (Ward 1)

Reading the results: on-time performance has improved year over year, and differences between wards are small. The large differences are between request types: routine, scheduled work is almost always on time, while repairs and inspections often miss their targets.

5. Key design and tool decisions

Decision	Why
SQLite with DB Browser instead of a database server	No server setup, and it imports the 188,406-row CSV in seconds. The SQL carries over to SQL Server or Azure SQL with small changes, mainly date functions.
Clean in SQL, keep the raw table untouched	Every step is visible and re-runnable, and the original data is always there to trace back to.
Turn empty strings into NULLs	CSV imports store blanks as empty text, so `IS NULL` checks would silently miss them.
Exclude, don't fix, the 915 impossible timestamps	The correct times can't be known from the data. Excluding them and reporting the count is more honest than guessing.
Count only closed requests in the on-time rate	A request still open can't be judged yet. Open requests already past target (2,689) are shown separately so they aren't hidden.
Require 100+ requests per request type	Prevents tiny categories (2 or 3 requests) from looking like extreme results.
Compare Jan–Sep across years	2024 starts in late February and 2026 ends in October, so full-year comparisons would be uneven.
Put business rules in one table (`requests_bi`)	The definition of "on time" lives in one place, so SQL and any BI tool give the same number.
Python (matplotlib) for the dashboard image	Reproducible from code and easy to version in GitHub. The `requests_bi` table is ready for Power BI if an interactive version is needed.
2026 wards	Matches the City's current boundaries and its public dashboard.

6. Most significant technical challenge

Problem: getting the data into a database with dates that could be used for math.

1. **Slow import.** MySQL Workbench's import wizard inserts rows one at a time, which was far too slow for 188,406 rows. *Resolution:* switched to SQLite, which imported the file in seconds.
2. **Messy raw file.** The first column name carried a hidden byte-order-mark character (`ï»¿OBJECTID`), several column names had spaces, blanks came in as empty text, and a few rows were missing trailing columns. *Resolution:* the cleaning script selects only the needed columns under new names, uses `TRIM` and `NULLIF`, and casts numeric fields.
3. **US-format dates.** Dates arrived as `10/23/2026 11:55:19 PM`. SQLite only does date math on ISO format (`2026-10-23 23:55:19`), and it has no built-in parser for the US format. Text sorting was also wrong (`9/1/2026` sorts after `10/1/2026`). *Resolution:* wrote a SQL `UPDATE` with CTEs that split each date using `instr` and `substr`, converts the 12-hour clock to 24-hour with `CASE`, and rebuilds the value with `printf`.

Validation of the fix: row count unchanged (188,406) and **0 rows** failing an ISO date-format check (`NOT LIKE '____-__-__ __:__:__'`).

7. How the project could be developed further

With more data or time	What it would add
The City's official on-time definition	Confirm or reconcile the 68.1% with the published measure
Open-request aging	Track the backlog of open, overdue requests over time, not just closed ones
Target-setting rules by request type	Separate "targets too tight" from "work takes too long" for low on-time types
Crew capacity and work-order data	Explain delays by staffing and workload
Weather and seasonality	Explain monthly swings (for example, potholes after freeze-thaw cycles)
Census data by ward	Check whether service timeliness differs by neighborhood characteristics
Automated pipeline	Daily refresh into Azure SQL with Azure Data Factory, published as an interactive Power BI report
Data-quality monitoring	Automatic alerts when impossible timestamps or missing targets appear

8. Limitations

- Public data only; the City's official method may differ.
- Target dates come from the source system and are not evaluated.
- Recent months can change as open requests close.
- The likely cause of the 915 impossible timestamps is a hypothesis, not confirmed.

9. Files

File	Purpose
`sql/01_clean_311.sql`	Clean the raw import
`sql/02_data_checks.sql`	Validation checks, with results noted
`sql/03_analysis.sql`	On-time analysis, with results noted
`sql/04_bi_table.sql`	Final table with business-rule flags
`python/dashboard.py`	Builds the dashboard image
`images/dashboard.png`	Dashboard
`docs/311_on_time_summary.pdf`	One-page summary of findings